

AI Specialization 4: Domain Independent Planning

AI Specialization 4: PUI

Domain independent planning: representations, heuristics, algorithms, nondeterministic and probabilistic planning, MDPs, and MCTS.

Sources used: official CTU CourseWare page for BE4M36PUI, which lists lectures on classical planning, state-space search, relaxation heuristics, landmarks/LM-cut, LP heuristics, abstraction heuristics, nondeterministic planning, probabilistic planning, MDPs/POMDPs, and MCTS; CTU state-exam topic list; standard automated planning material.

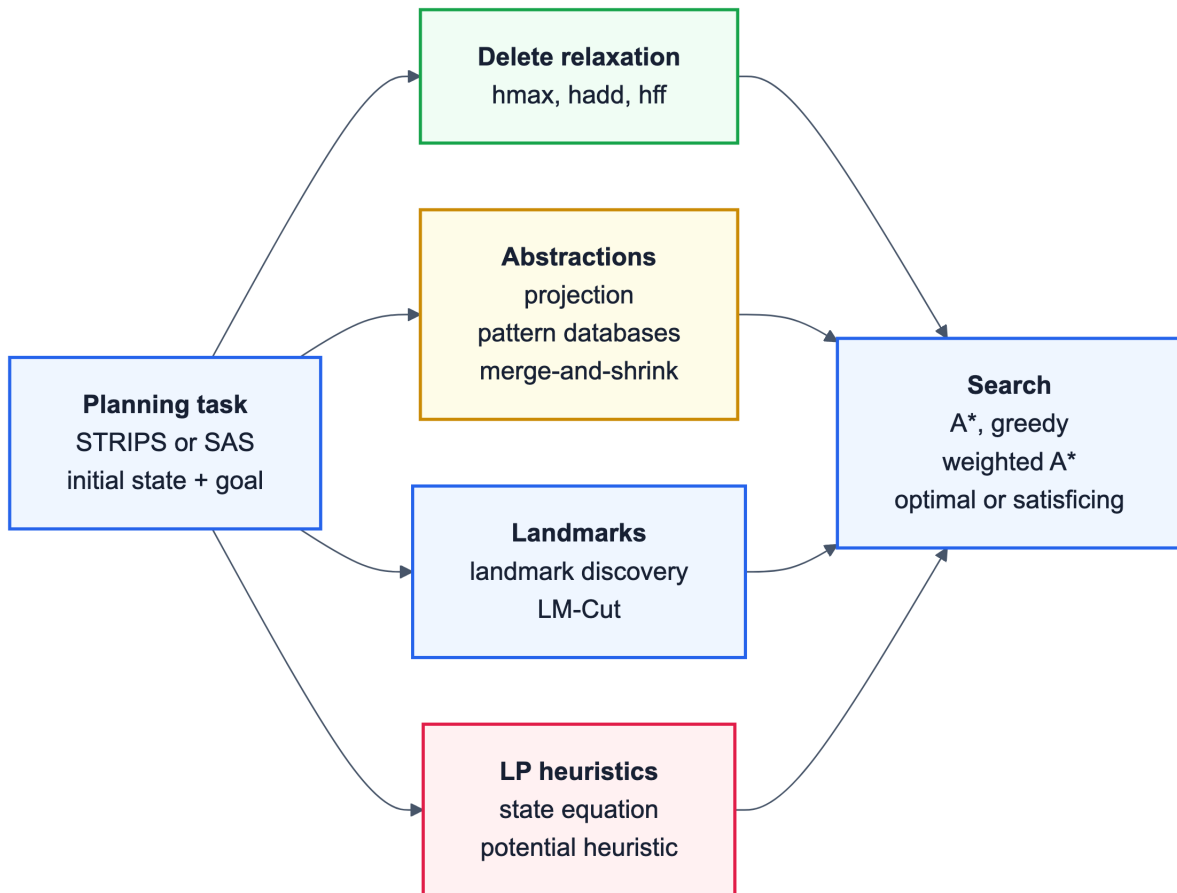


Figure 1: Planning heuristic families: different relaxations and abstractions estimate distance-to-goal and guide heuristic search.

PUI in Plain English

PUI is about planning actions before acting. The main trick is to estimate distance to the goal using relaxations, abstractions, landmarks, or optimization.

1. State the planning model.

States, actions, initial state, goal.

2. Choose the heuristic.

Relaxed, abstracted, landmark, LP, or sampled.

3. Say the guarantee.

Satisficing, optimal, admissible, probabilistic.

Classical planning

Find an action sequence from initial state to goal.

- STRIPS: facts, preconditions, add/delete effects.
- SAS: finite-domain variables.
- A*, greedy search, and heuristic search are central.

Delete relaxation

Pretend actions never delete facts.

- hmax: optimistic max cost.
- hadd: sums subgoal costs, may double count.
- hff: extracts a relaxed plan.

Abstractions and landmarks

Simplify the problem or identify required facts/actions.

- Pattern databases solve projected tasks.
- Merge-and-shrink builds abstract transition systems.
- Landmarks must be achieved on every plan.

LP heuristics

Use linear constraints to lower-bound plan cost.

- State-equation heuristic counts action occurrences.
- Potential heuristics assign values to facts/states.
- LP gives a numeric estimate for search.

Uncertainty and MCTS

Actions may be nondeterministic or probabilistic.

- MDP value iteration optimizes expected return.
- MCTS samples futures instead of expanding all states.
- UCT balances value and exploration.

How to answer

Always say what information is ignored or approximated.

- What is the relaxed problem?
- Is the heuristic admissible?
- What search uses it?

Exam sentence: Delete-relaxation heuristics estimate goal distance by solving an easier problem where delete effects are ignored, so facts only accumulate.

Three traps

- Satisficing means "find some plan"; optimal means minimum cost.
- hadd is usually not admissible because it double-counts shared actions.
- MCTS is useful when the full search tree is too large to enumerate.

Use this page to remember what each heuristic family is simplifying.

1. Planning Tasks and Heuristic Search

Classical planning

A classical planning task assumes:

- finite set of states,
- deterministic actions,
- fully observable state,
- known initial state,
- goal condition,
- static domain.

A plan is a sequence of actions transforming the initial state into a goal state.

STRIPS representation

STRIPS uses propositional facts. Task:

$$\Pi = (F, A, I, G)$$

where:

- F facts,
- A actions,
- $I \subseteq F$ initial facts,
- $G \subseteq F$ goal facts.

Action a has:

- preconditions $pre(a)$,
- add effects $add(a)$,
- delete effects $del(a)$.

Action applicable in state s iff $pre(a) \subseteq s$. Result:

$$\gamma(s, a) = (s \setminus del(a)) \cup add(a).$$

SAS representation

SAS uses finite-domain variables:

$$V = \{v_1, \dots, v_n\}, \quad D(v_i) \text{ finite.}$$

A state assigns one value to every variable. Actions have partial variable assignments as preconditions/effects.

SAS can be more compact than STRIPS because mutually exclusive facts become values of one variable.

Satisficing vs. optimal planning

- **Satisficing planning:** find any valid plan, usually fast.
- **Optimal planning:** find plan with minimum cost or length; requires admissible search to prove optimality.

Heuristic search

Planning is often modeled as graph search in state space.

Common algorithms:

- BFS for unit costs,
- uniform-cost search for positive costs,
- greedy best-first search with priority $h(s)$,
- A* with priority $f(s) = g(s) + h(s)$.

Heuristic properties

A heuristic $h(s)$ estimates cost from state s to goal.

- **Admissible:** $h(s) \leq h^*(s)$ for all s .
- **Consistent:** $h(s) \leq c(s, a, s') + h(s')$ for every transition.
- **Safe/dead-end aware:** returns ∞ for recognized dead ends.
- **Informative:** closer to h^* is usually better.

A* with admissible heuristic is optimal; consistency avoids reopening closed nodes.

Simple explanation

A planning heuristic is like a GPS estimate of remaining distance. If it never overestimates, A* can trust it for optimal planning. If it is too optimistic but informative, it can still be excellent for quickly finding some plan.

2. Delete Relaxation and Abstraction Heuristics

Delete relaxation

Delete relaxation ignores delete effects:

$$del(a) = \emptyset.$$

The relaxed problem is easier because facts only accumulate. It may allow impossible plans, so its optimal cost is an admissible lower bound for the original task.

h^{max}

Estimate cost of achieving facts:

$$h^{max}(s, p) = \begin{cases} 0 & p \in s, \\ \min_{a: p \in add(a)} \left(cost(a) + \max_{q \in pre(a)} h^{max}(s, q) \right) & otherwise. \end{cases}$$

Goal value:

$$h^{max}(s) = \max_{g \in G} h^{max}(s, g).$$

Admissible but often weak because it assumes subgoals can be achieved together for the maximum cost.

h^{add}

Use sum instead of maximum:

$$h^{add}(s, p) = \min_{a: p \in add(a)} \left(cost(a) + \sum_{q \in pre(a)} h^{add}(s, q) \right).$$

Goal value:

$$h^{add}(s) = \sum_{g \in G} h^{add}(s, g).$$

More informative but not admissible because it double-counts shared actions.

h^{ff}

Fast-Forward heuristic:

1. build relaxed planning graph,
2. extract one relaxed plan greedily backward from goals,
3. heuristic is number/cost of actions in that relaxed plan.

h^{ff} is not admissible, but works well in satisficing planning.

Simple explanation

Delete relaxation imagines that actions never undo facts. That fantasy world is easier: once a door is open, it stays open; once a package is loaded, it can still magically be somewhere else later. The resulting plan may be impossible, but its cost is useful guidance.

Example

In a logistics task, moving a truck from city A to city B normally deletes `at(truck,A)` and adds `at(truck,B)`. Delete relaxation keeps both facts. This can make relaxed plans overly optimistic, but it quickly estimates which actions are relevant.

Abstraction heuristics

An abstraction maps concrete states to abstract states:

$$\alpha : S \rightarrow S^\alpha.$$

Solve the abstract problem exactly and use abstract distance:

$$h^\alpha(s) = \text{dist}^\alpha(\alpha(s), G^\alpha).$$

If abstraction does not increase costs, h^α is admissible.

Projection and pattern databases

Projection abstraction keeps only selected variables (pattern P). The pattern database stores exact goal distances for all abstract states over P .

Advantages:

- admissible,
- very fast lookup after preprocessing.

Tradeoff: larger patterns are more informative but exponentially larger.

Merge & Shrink

Merge-and-shrink abstraction:

1. create atomic abstractions for variables,
2. iteratively merge abstractions,
3. shrink abstract state spaces to keep size manageable,
4. compute distances in final abstraction.

It can produce strong admissible heuristics while controlling memory.

3. Landmarks and LM-Cut

Landmarks

A landmark is a fact, action, or formula that must be true/achieved in every valid plan.

Example: if goal is `at(package, destination)`, then loading the package may be an action landmark in logistics-like domains.

Landmark orders:

- natural order: L_1 must be achieved before L_2 ,
- greedy necessary order: L_1 must hold immediately before first achieving L_2 ,
- reasonable order: not logically necessary but useful.

Landmark discovery

Common methods:

- relaxed planning graph analysis,
- backchaining from goals through achievers,
- checking necessary predecessors,
- disjunctive landmarks when one of several facts/actions must occur.

Landmarks can guide search by counting unachieved landmarks or by preferred operators.

Landmark heuristic

A simple landmark-count heuristic:

$$h(s) = \text{number or cost of landmarks not yet achieved from } s.$$

Must avoid double-counting and handle landmarks that can be deleted and re-achieved.

LM-Cut heuristic

LM-Cut is an admissible heuristic based on repeated cuts in a relaxed planning graph.

High-level algorithm:

1. Compute h^{max} values.
2. Identify a cut: a set of actions that every relaxed plan must use to cross from reachable facts to goal-relevant facts.
3. Let cut cost be minimum action cost in the cut.
4. Add this cost to heuristic.
5. Subtract cut cost from all actions in the cut.
6. Repeat until relaxed goal cost is zero.

LM-Cut is admissible and often much stronger than h^{max} .

Example

If every relaxed plan must use one of three actions to first achieve a required landmark, those actions form a cut. If their costs are 2, 5, and 7, LM-Cut can safely add 2 to the heuristic and subtract 2 from all actions in that cut.

Exam tip

Remember the roles: h^{max} gives a cheap admissible estimate, h^{ad} is usually stronger but not admissible, h^{ff} extracts one relaxed plan, and LM-Cut repeatedly finds necessary relaxed-action cuts to stay admissible.

4. Linear Programming Heuristics

State-equation heuristic

Represent a relaxed plan by action-count variables $x_a \geq 0$. For each fact/variable-value, require that action effects can support net changes needed from state to goal.

Generic form:

$$Mx \geq b(s, G), \quad x \geq 0.$$

Minimize action cost:

$$h(s) = \min \sum_a cost(a)x_a.$$

The LP relaxation gives an admissible heuristic if constraints are necessary for real plans.

Potential heuristic

Assign potentials to facts or variable values. For state s :

$$h(s) = \sum_{p \in s} w_p$$

or for SAS:

$$h(s) = \sum_i w_{i, s_i}.$$

Weights are chosen so that for every action:

$$h(s) \leq cost(a) + h(\gamma(s, a)),$$

and goal states have heuristic ≤ 0 . This makes h admissible and consistent. Weights can be optimized by LP to maximize heuristic values over relevant states.

Planning portfolios

Many planners combine several heuristics/search strategies. A portfolio may run planners sequentially or in parallel with time budgets. This is practical because planner performance is highly domain-dependent.

5. Nondeterministic and Probabilistic Planning

Nondeterministic planning

Actions have several possible outcomes:

$$\gamma(s, a) \subseteq S.$$

A solution is generally a policy, not a sequence:

$$\pi : S \rightarrow A.$$

Plan types:

- **weak plan:** at least one execution reaches goal.
- **strong plan:** all executions reach goal in finite time.
- **strong cyclic plan:** all fair executions reach goal, allowing loops while progress is eventually possible.

Simple explanation

In nondeterministic planning, a sequence of actions is usually not enough because the world may choose different outcomes. A policy says what to do after each possible outcome, like a contingency plan.

Algorithms use AND/OR search:

- OR nodes: planner chooses action.
- AND nodes: environment chooses outcome.

Probabilistic planning and MDPs

Probabilistic planning assigns probabilities to outcomes:

$$P(s'|s, a).$$

This is an MDP:

$$(S, A, P, R, \gamma).$$

Goal: find policy maximizing expected discounted reward or minimizing expected cost.

Value iteration

For discounted MDP:

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_k(s')].$$

For stochastic shortest-path planning, terminal goal states have value 0 and costs are minimized.

POMDPs

Partially observable MDPs maintain belief states $b(s)$, distributions over states. Actions update beliefs using Bayes rule after observations. POMDPs are much harder than fully observable MDPs.

Monte-Carlo Tree Search

MCTS builds an asymmetric search tree by simulation.

Each iteration:

1. **Selection:** descend tree using a tree policy.
2. **Expansion:** add a new node.
3. **Simulation:** roll out a default policy.
4. **Backpropagation:** update visit counts and values.

UCT selection rule:

$$a = \arg \max_a \left(\bar{X}(s, a) + C \sqrt{\frac{\ln N(s)}{N(s, a)}} \right).$$

First term exploits high-value actions; second explores uncertain actions. UCT is widely used in games and planning with generative models.

Example

Suppose action A has average rollout value 0.7 from 100 visits, while action B has 0.6 from only 4 visits. UCT may choose B because the exploration bonus for 4 visits is much larger, so B is still uncertain and worth testing.

Exam Checklist

- Define STRIPS and SAS and compare them.
- Explain satisficing vs. optimal planning.
- State admissible/consistent heuristic properties.
- Compute/describe h^{max} , h^{add} , and h^{ff} .
- Explain abstraction, projection, pattern databases, and merge-and-shrink.
- Define landmarks, landmark orders, and LM-Cut.
- Explain state-equation and potential heuristics.
- Distinguish weak, strong, and strong cyclic nondeterministic plans.
- Define MDP value iteration and MCTS/UCT.